

Apache Airflow

Document ID

DOC011

Project

Apache Airflow

Category

Data Engineering

Batch

batch_4

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

Airflow Overview

Apache Airflow is a workflow orchestration platform used to define, schedule, and monitor data workflows. A workflow is represented as a Directed Acyclic Graph, or DAG, where tasks describe individual units of work and dependencies describe the order in which those tasks should run. Airflow is commonly used in data engineering to coordinate ingestion, transformation, validation, and downstream processing without embedding the business logic inside the scheduler itself.

DAGs, Tasks, and Dependencies

A DAG defines the structure of a workflow. Tasks may run Python functions, execute SQL, trigger external jobs, or call other systems through operators and hooks. Dependencies such as `task_a >> task_b` make the execution order explicit. This is useful for pipelines where a transformation must wait for ingestion to finish, or a validation step must succeed before data is published. Clear dependencies make failures easier to isolate and reruns easier to manage.

Scheduling and Reruns

Airflow can run workflows on a schedule or through manual triggers. Each scheduled run has its own execution context, which helps teams reason about daily or hourly processing windows. Data pipelines should still be designed to be idempotent, because orchestration tools may retry failed tasks or operators may rerun a historical execution. A safe rerun should not create duplicate data or corrupt previously successful output.

Retries, Monitoring, and Operational Visibility

Tasks can be configured with retries, retry delays, and failure handling. Airflow records task states and workflow history, allowing engineers to inspect which task failed and when it failed. Logs provide operational visibility, while dependencies ensure that downstream tasks do not continue after an upstream failure unless the workflow explicitly allows it. This separation between orchestration and transformation logic keeps data workflows easier to maintain.

Airflow in Data Engineering

Airflow is most useful when a pipeline contains multiple independent systems or steps that must be coordinated. For example, a workflow may ingest files, run a Spark transformation, validate row counts, and then refresh a downstream table. Airflow does not replace the processing engines that perform those operations; it coordinates when and in what order they run. This distinction is important when designing modular data platforms.